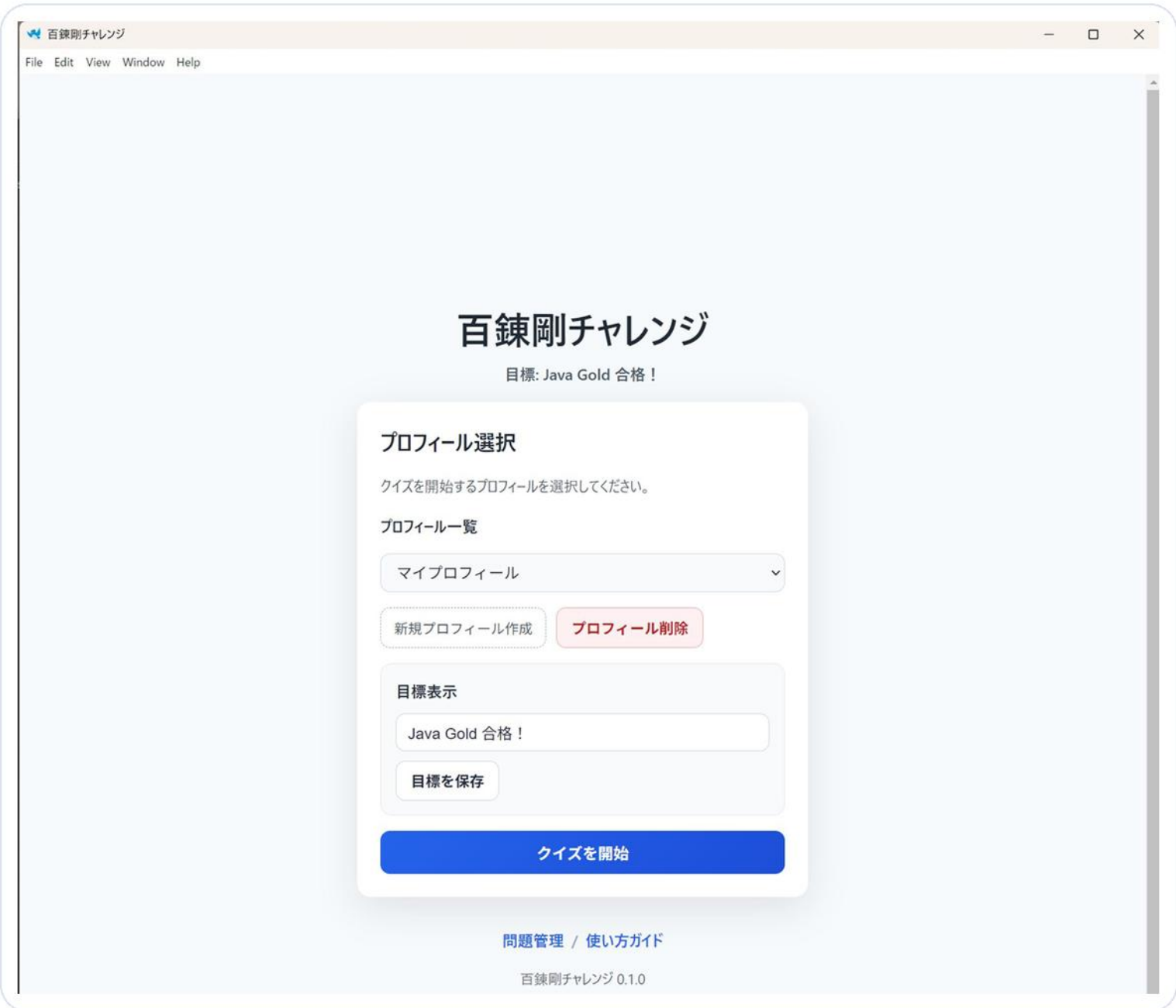
百錬剛チャレンジ
利用ガイド

クイズ学習、学習履歴の確認、問題管理の使い方をまとめたガイドです。

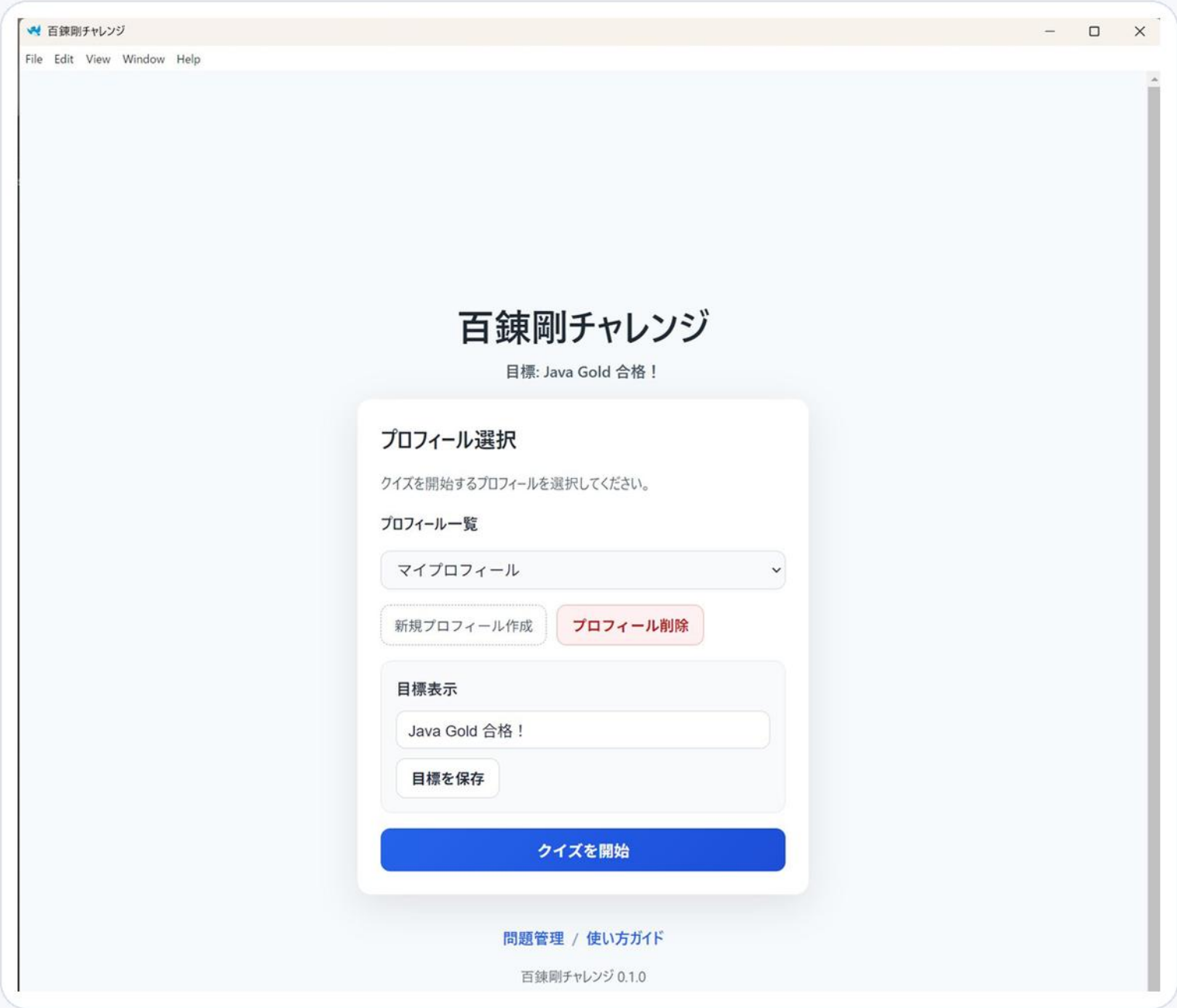
- 初回起動時は画面表示まで数秒かかることがあります。
- データは通常、AppData 配下の quiz_database.db に保存されます。
- 詳しいバックアップ手順は本ガイド末尾の補足を参照してください。



作成日: 2026-03-29

1. クイズの始め方

最初にプロフィールを選び、学習目標を設定してからクイズを開始します。

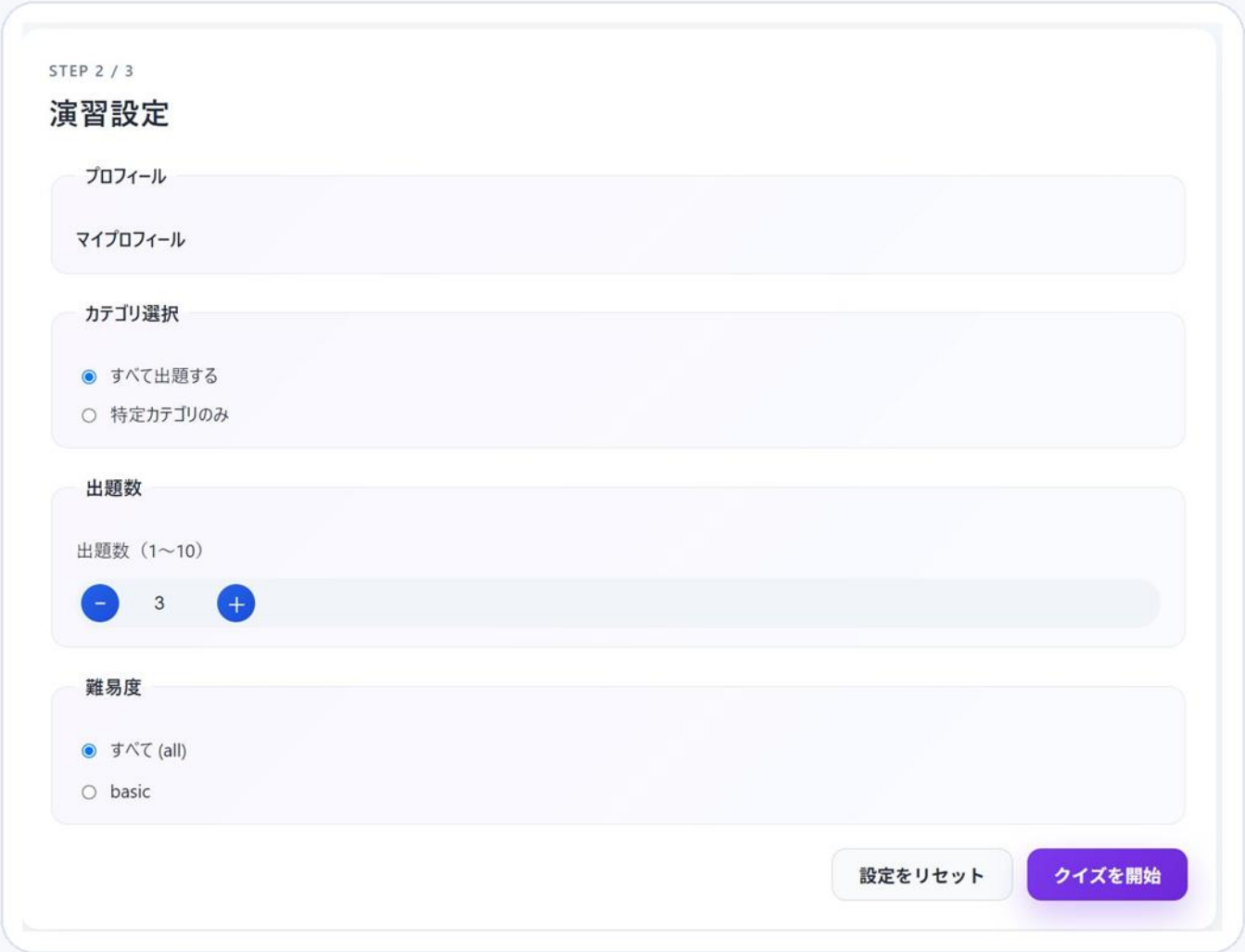


操作ポイント

- プロフィール一覧から利用する名前を選びます。
- 必要なら「新規プロフィール作成」で利用者を追加し、「プロフィール削除」で不要なプロフィールを整理できます。
- 目標表示に学習目標を入力して保存すると、上部に常時表示されます。
- 「クイズを開始」を押すと演習設定画面へ進みます。

2. 演習設定

出題カテゴリ、問題数、難易度を設定して、今回の演習範囲を決めます。

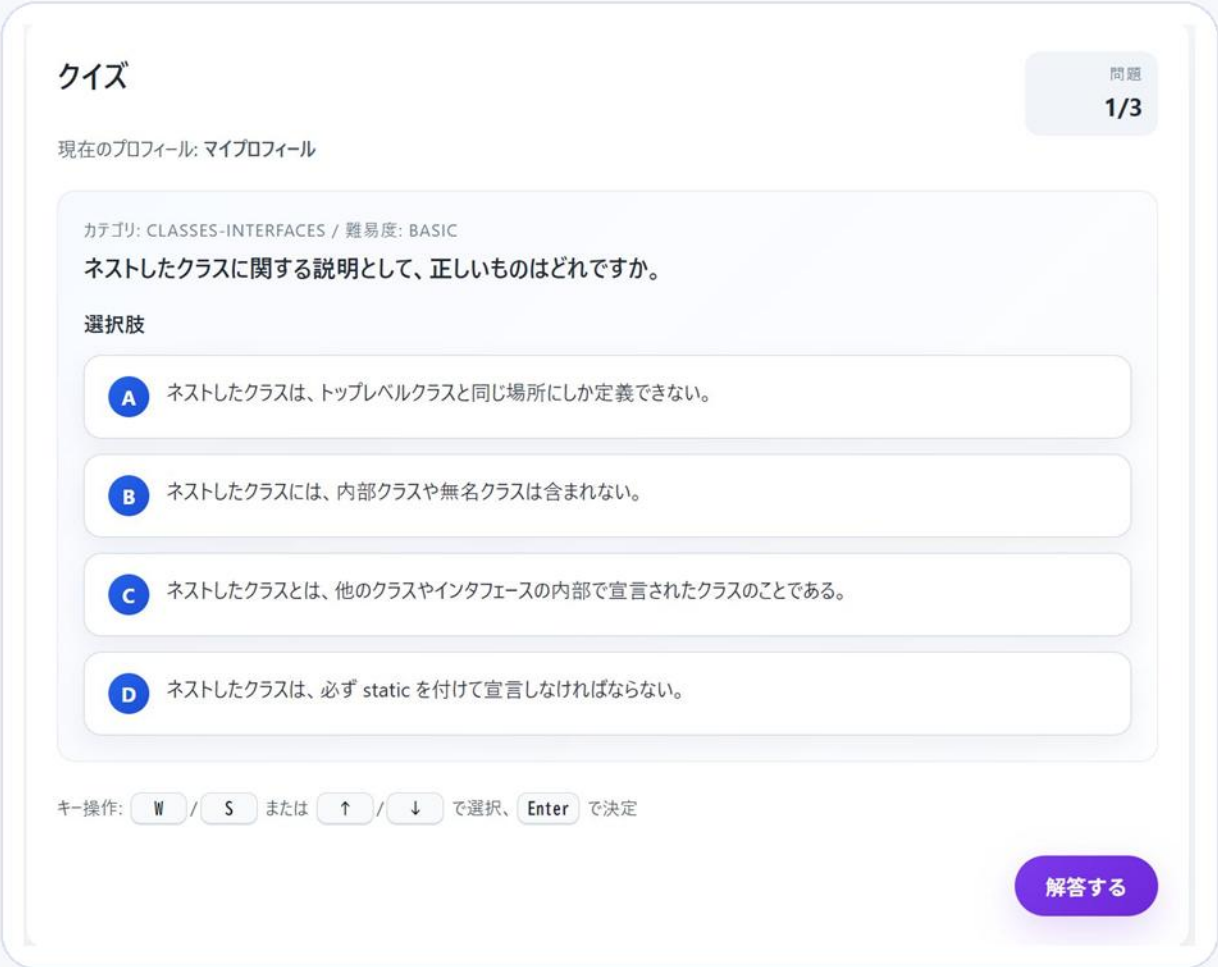


操作ポイント

- カテゴリは「すべて出題する」または「特定カテゴリのみ」から選択できます。
- 出題数は画面のボタンで増減します。
- 難易度は `all` や `basic` などから選択できます。
- 設定内容をやり直す場合は「設定をリセット」、開始する場合は「クイズを開始」を押します。

3. クイズ画面

設問を読み、選択肢を選んで回答します。コード付き問題にも対応しています。



操作ポイント

- 画面右上で現在の問題番号を確認できます。
- 選択肢はクリックで選べます。キーボード操作は `W` / `S` または上下キーで移動し、`Enter` で確定します。
- 文章問題だけでなく、コードスニペット付き問題も出題されます。

4. コード付き問題

コードスニペット付きの問題では、表示された Java コードを読みながら回答します。

操作ポイント

- コードは専用の表示エリアに整形されて表示されます。
- 選択肢の操作方法は通常のクイズ画面と同じです。
- ストリーム API や並列処理など、コード読解を含む問題の学習に向いています。

5. 結果サマリー

演習終了後は、正解数と各問題の振り返りを一覧で確認できます。

操作ポイント

- 画面上部にスコアと正解数が表示されます。
- 各問題ごとに、自分の回答と正解を見比べられます。
- 「解説を見る」を開くと、解説や学習ポイントをその場で確認できます。
- 「カテゴリ選択に戻る」で次の演習設定へ戻れます。

6. 学習ダッシュボード

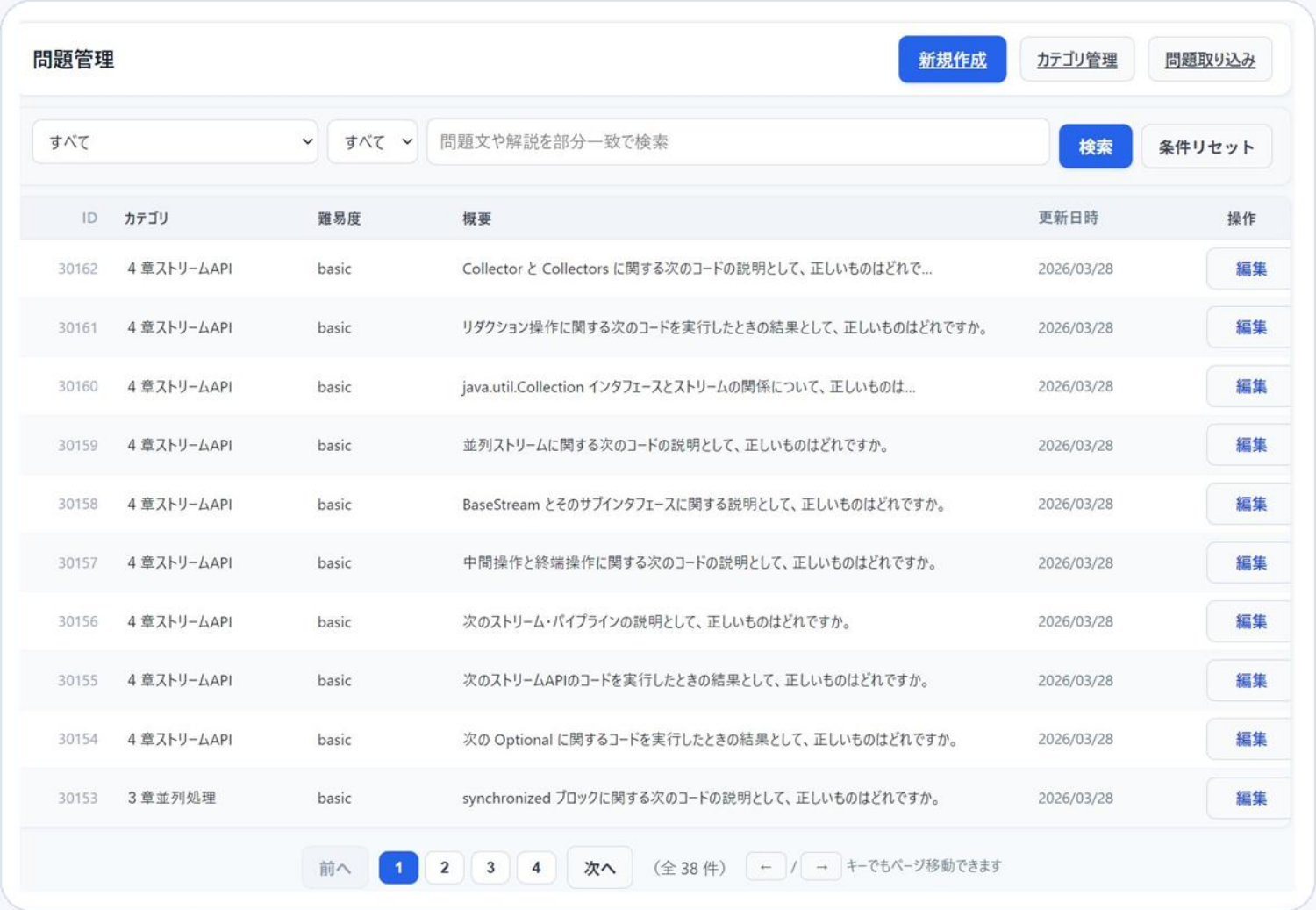
プロフィールごとの学習状況を数値で確認し、次の学習方針の参考にします。

操作ポイント

- 総解答数、正答率、習得済み、弱点などの主要指標を確認できます。
- 学習アドバイス欄には、次に優先したい学習テーマが表示されます。
- 定期的に確認すると、苦手分野の把握に役立ちます。

7. 問題管理とカテゴリ管理

問題一覧の検索や編集、カテゴリの表示名・並び順・利用可否の管理ができます。

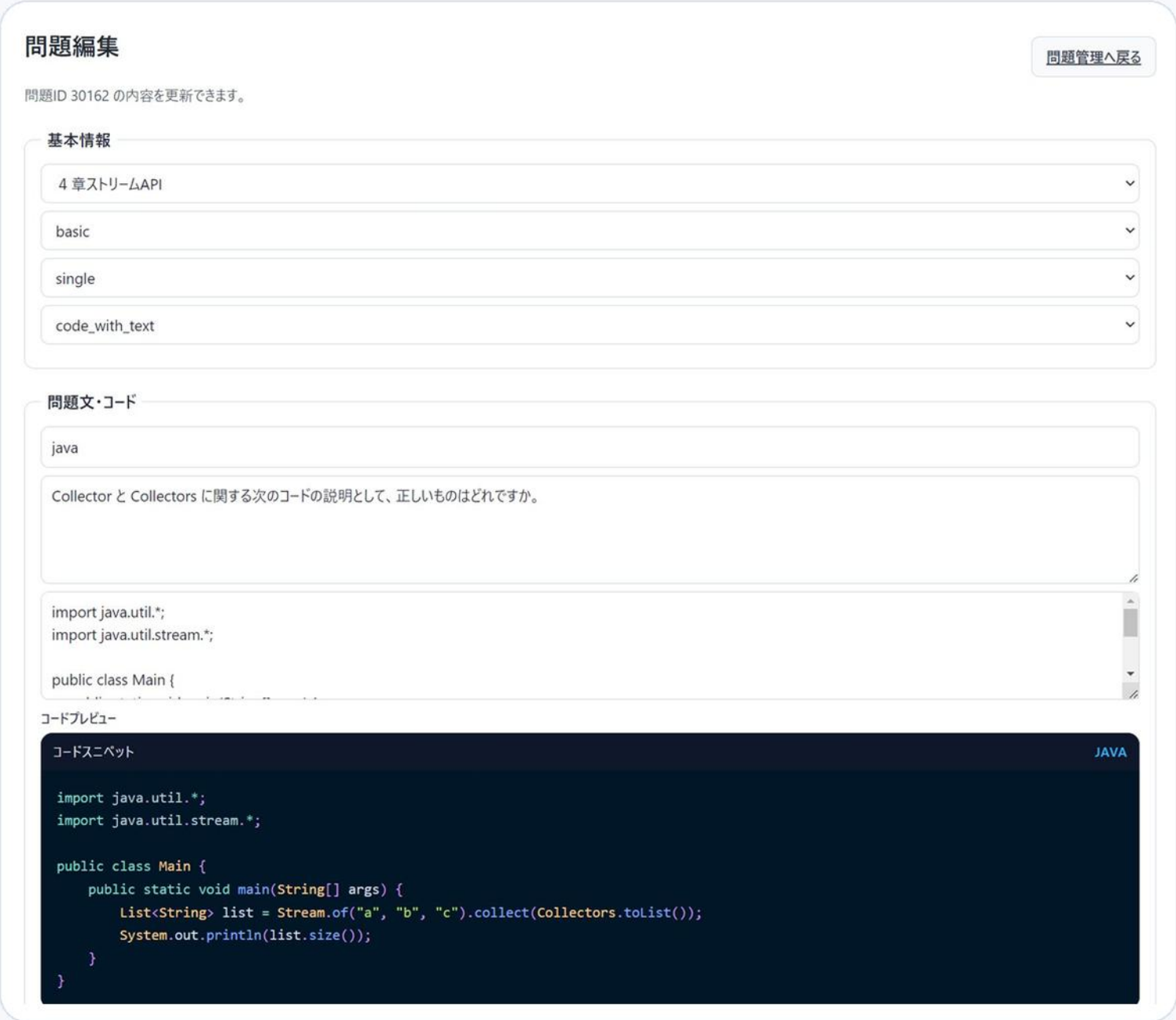


操作ポイント

- 問題管理ではカテゴリ・難易度・キーワードで一覧を絞り込みできます。
- 「新規作成」から問題追加、「編集」から既存問題の修正ができます。
- 「カテゴリ管理」では表示名、並び順、有効状態を調整できます。

8. 問題編集

問題編集画面では、問題文、コード、選択肢、解説などを細かく更新できます。

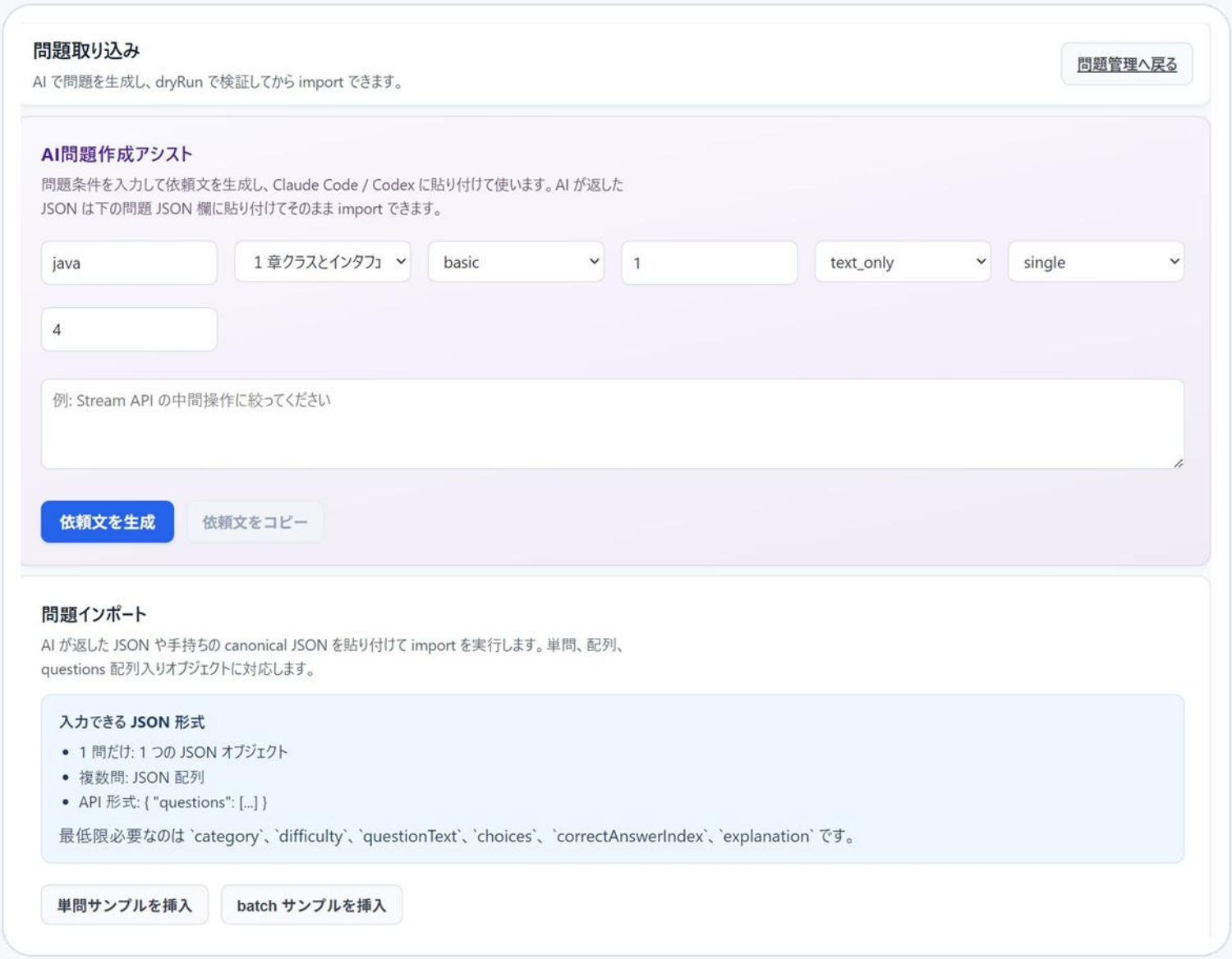


操作ポイント

- カテゴリ、難易度、出題形式などの基本情報を上部で設定します。
- 問題文やコードは入力欄で編集し、下部のコードプレビューで見え方を確認できます。
- 選択肢や正解、解説、学習ポイントもここで一括管理します。

9. 問題取り込み

AI で作成した問題 JSON や手元の JSON を使って、大量の問題をまとめて登録できます。

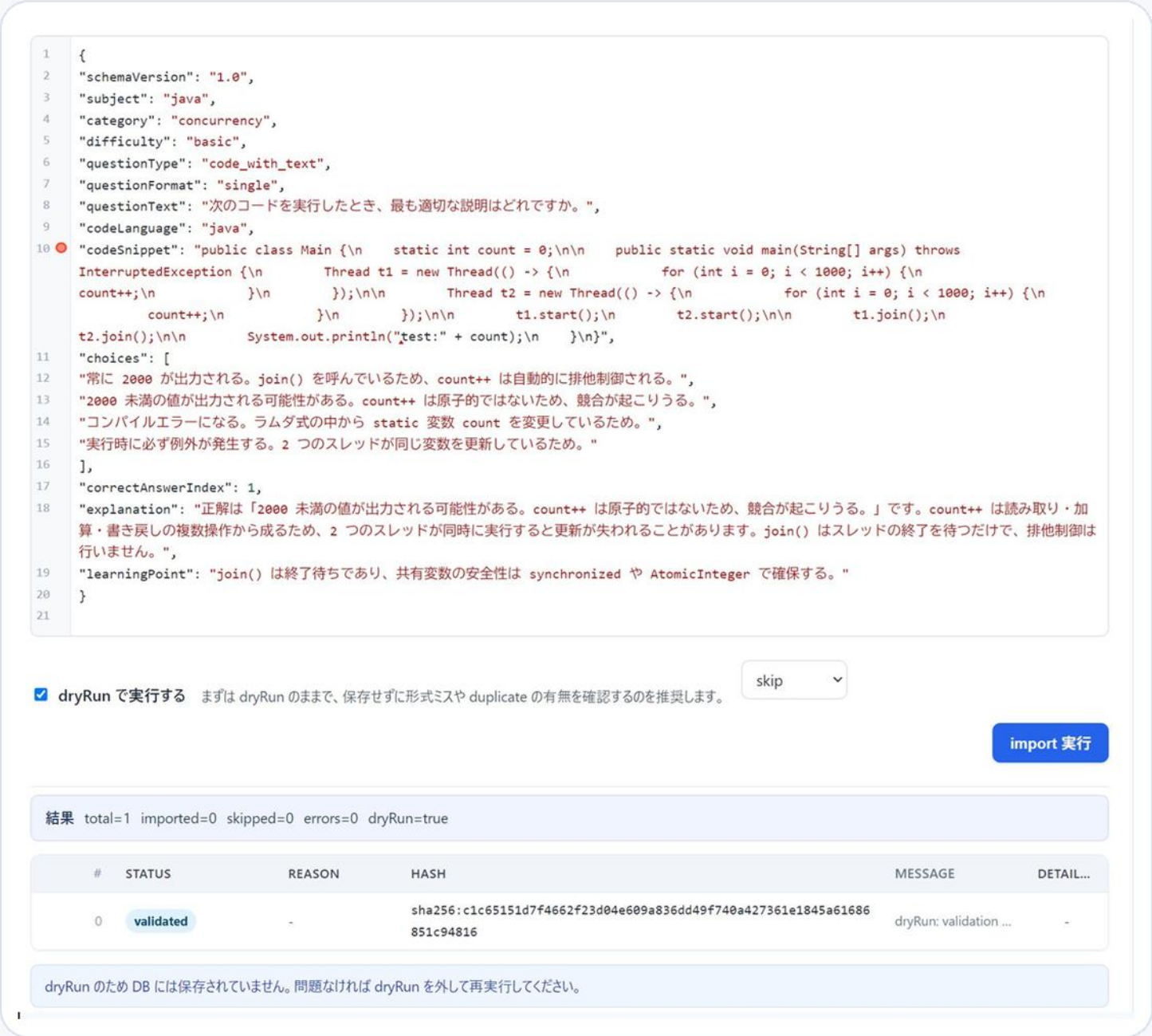


操作ポイント

- 上部の AI 問題作成アシストでは、科目、カテゴリ、難易度、形式、件数を指定して依頼文を生成できます。
- 生成した依頼文は Claude Code や Codex に貼り付けて使う想定です。
- 下部の問題インポート欄には、単問 JSON、配列 JSON、`questions` 配列入りオブジェクトを貼り付けられます。

10. dryRun での検証

インポート前に `dryRun` で検証すると、保存せずに形式チェックや重複確認ができます。



```
{
"schemaVersion": "1.0",
"subject": "java",
"category": "concurrency",
"difficulty": "basic",
"questionType": "code_with_text",
"questionFormat": "single",
"questionText": "次のコードを実行したとき、最も適切な説明はどれですか。",
"codeLanguage": "java",
"codeSnippet": "public class Main {\n    static int count = 0;\n\n    public static void main(String[] args) throws
InterruptedException {\n        Thread t1 = new Thread(() -> {\n            for (int i = 0; i < 1000; i++) {\n
count++;\n            }\n        });\n\n        Thread t2 = new Thread(() -> {\n            for (int i = 0; i < 1000; i++) {\n
        count++;\n            }\n        });\n\n        t1.start();\n        t2.start();\n\n        t1.join();\n
t2.join();\n\n        System.out.println("test:" + count);\n    }\n}",
"choices": [
"常に 2000 が出力される。join() を呼んでいるため、count++ は自動的に排他制御される。",
"2000 未満の値が出力される可能性がある。count++ は原子的ではないため、競合が起こりうる。",
"コンパイルエラーになる。ラムダ式の中から static 変数 count を変更しているため。",
"実行時に必ず例外が発生する。2 つのスレッドが同じ変数を更新しているため。"
],
"correctAnswerIndex": 1,
"explanation": "正解は「2000 未満の値が出力される可能性がある。count++ は原子的ではないため、競合が起こりうる。」です。count++ は読み取り・加算・書き戻しの複数操作から成るため、2 つのスレッドが同時に実行すると更新が失われることがあります。join() はスレッドの終了を待つだけで、排他制御は行いません。",
"learningPoint": "join() は終了待ちであり、共有変数の安全性は synchronized や AtomicInteger で確保する。"
}
```


操作ポイント

- まず `dryRun` を有効にしたまま `import 実行` し、エラーや重複の有無を確認します。
- 結果一覧には `validated` や `skipped` などの状態が表示されます。
- 問題がなければ `dryRun` を外して再実行し、本登録します。

補足情報

データ保存場所

アプリで利用するデータベースは、通常次の場所に保存されます。
C:¥Users¥<Windowsユーザー名>¥AppData¥Roaming¥java-gold-quiz¥data¥quiz_database.db

バックアップと復元

- バックアップする場合は、アプリを終了した状態で `quiz_database.db` を別フォルダへコピーします。
- 復元する場合は、アプリを終了した状態で同じ場所に上書きコピーします。
- 同じ PC でも Windows ユーザーが異なるとデータは別々に保存されます。
- 過去データが残っている場合は、保存済みデータが優先して読み込まれます。

困ったとき

- 起動直後に画面が出ない場合は数秒待ち、それでも表示されないときはアプリと `java.exe` を終了して再起動してください。
- JSON 取り込みは、まず `dryRun` のまま実行して検証してから本登録するのが安全です。

このガイドに載っている画面は作成時点のものです。

表示や文言が一部変わっていても、基本的な流れは同じです。